

BACSE202 – Database Systems

Module 4: Transaction Processing – Quick Exam Notes

Covers slides 1–90 of BACSE202-DBS-Module-4.ppt (Elmasri & Navathe Ch. 21; Silberschatz Ch. 14)

Contents

1	Transactions – Basics (Slides 3–14)	1
2	Why Concurrency Control is Needed (Slides 15–19)	2
3	Why Recovery is Needed (Slides 20–23)	2
4	ACID Properties (Slides 24–31)	2
5	Transaction States (Slides 32–35)	3
6	Concurrent Executions and Schedules (Slides 36–46)	3
7	Schedules Based on Recoverability (Slides 47–55)	4
8	Schedules Based on Serializability (Slides 56–67)	5
9	Testing Conflict Serializability – Precedence Graph (Slides 68–72)	6
10	View Serializability and Other Equivalences (Slides 73–79)	6
11	Transaction Support in SQL (Slides 80–86)	7
12	Serializability – Silberschatz View (Slides 87–90)	7
13	One-Page Revision	8

1 Transactions – Basics (Slides 3–14)

Definition

A **transaction** is a *unit of program execution* that accesses and possibly updates data items – a **logical unit of database processing**.

- **Main operations:** *read* (retrieval, e.g. SQL SELECT) and *write* (modify, e.g. INSERT, UPDATE, DELETE).
- Example: transfer \$100 from a checking account to a savings account.
- A transaction is either **stand-alone** (SQL typed interactively) or **embedded** in a program (most transactions).
- **Transaction processing systems:** large multi-user systems running thousands of concurrent transactions per minute (banking, airline reservations).

Two modes of concurrency (Fig. 21.1)

- **Interleaved processing:** one CPU; operations of transactions A and B are interleaved over time (t_1 – t_2).
- **Parallel processing:** multiple CPUs; C runs on CPU₁ while D runs on CPU₂ (t_3 – t_4).
- Basic transaction theory assumes **interleaved** concurrency.

Basic operations on a data item X

<code>read_item(X)</code>	1. Find the address of the disk block containing X . 2. Copy that block into a main-memory buffer (if not already there). 3. Copy X from the buffer into program variable X .
<code>write_item(X)</code>	1. Find the block address. 2. Copy the block into a buffer (if not there). 3. Copy program variable X into its place in the buffer. 4. Store the updated block back to disk – <i>immediately or later</i> .

Operations tracked by the Recovery Manager

1. **begin_transaction** – start of execution.
2. **read / write** – operations on items.
3. **end_transaction** – reads/writes are over; now check whether changes can be *committed* or the transaction must be *aborted* (e.g. violates concurrency control).
4. **commit_transaction** – successful end; changes are permanent.
5. **rollback / abort** – unsuccessful end; changes must be undone.

Notation (Fig. 21.2)

T_1	T_2
<code>read_item(X);</code>	<code>read_item(X);</code>
<code>$X := X - N;$</code>	<code>$X := X + M;$</code>
<code>write_item(X);</code>	<code>write_item(X);</code>
<code>read_item(Y);</code>	
<code>$Y := Y + N;$</code>	
<code>write_item(Y);</code>	

Shorthand:

$T_1 : b_1; r_1(X); w_1(X); r_1(Y); w_1(Y); e_1;$

$T_2 : b_2; r_2(X); w_2(X); e_2;$

b_i, e_i = begin/end; i = transaction id. Later also c_i (commit), a_i (abort).

2 Why Concurrency Control is Needed (Slides 15–19)

Problem	What happens
Lost Update (Fig. 21.3a)	Two transactions read the <i>same original</i> value of X and both update it; one update overwrites the other.
Temporary Update / Dirty Read (Fig. 21.3b)	T_1 writes X , T_2 reads that X , then T_1 <i>fails</i> and X is rolled back – T_2 has used a value that “never existed”.
Incorrect Summary (Fig. 21.3c)	An aggregate (e.g. sum of balances) reads some values <i>before</i> and others <i>after</i> another transaction’s update.
Unrepeatable Read	T_1 reads X twice and gets different values because T_2 changed X in between (e.g. seats on a flight).

Interleavings from Fig. 21.3

(a) Lost update

T_1	T_2
$r(X); X := X - N$	
$w(X); r(Y)$	$r(X); X := X + M$
$Y := Y + N; w(Y)$	$w(X) \leftarrow \text{lost}$

(a) T_1 ’s write of X is overwritten by T_2 , which used the old value.

(b) Temporary update / dirty read

T_1	T_2
$r(X); X := X - N$	
$w(X)$	
$r(Y)$ <i>fails</i>	$r(X); X := X + M$
	$w(X)$

(b) T_1 fails and must restore the old X , but T_2 already read the *temporary* value.

(c) Incorrect summary

T_1	T_3
$r(X); X := X - N; w(X)$	$sum := 0; r(A); sum := sum + A; \dots$
	$r(X); sum := sum + X$
$r(Y); Y := Y + N; w(Y)$	$r(Y); sum := sum + Y$

(c) T_3 reads X *after* N is subtracted but Y *before* N is added, so the sum is off by N .

3 Why Recovery is Needed (Slides 20–23)

Causes of transaction failure:

1. **Computer failure (system crash)** – hardware/software error; main memory contents may be lost.
2. **Transaction or system error** – integer overflow, division by zero, bad parameters, logical error, user interrupt.
3. **Local errors / exception conditions** – e.g. data not found, insufficient balance $\Rightarrow$ programmed abort.

4. **Concurrency control enforcement** – CC aborts the transaction (violates serializability or deadlock); restarted later.
5. **Disk failure** – blocks lost due to read/write malfunction or head crash.
6. **Physical problems & catastrophes** – power/AC failure, fire, theft, sabotage, overwriting disks, wrong tape mounted.

Failures 1–4 are common; 5–6 are more severe (need backup/archive).

System operations during recovery:

- **undo(X)** – like rollback, but for a *single write* operation.
- **redo(X)** – re-apply a write of a *committed* transaction to ensure it is permanent on disk.

4 ACID Properties (Slides 24–31)

Running example – transfer \$50 from *A* to *B*:

1. *read(A)* 2. *A := A - 50* 3. *write(A)* 4. *read(B)* 5. *B := B + 50* 6. *write(B)*

Two main issues: **failures** (hardware, crashes) and **concurrent execution**.

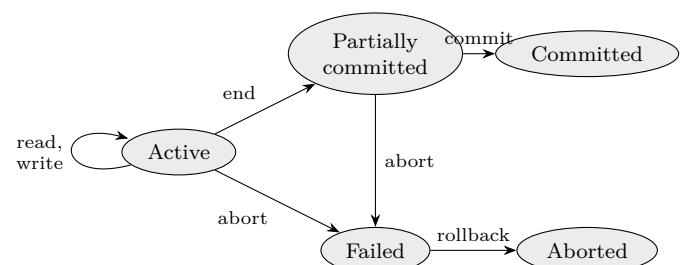
Property	Meaning	In the \$50 transfer
Atomicity	All operations are reflected in the DB, or none.	If failure after step 3 but before step 6, \$50 is “lost”; partial updates must not be reflected.
Consistency	A transaction run in isolation takes the DB from one consistent state to another.	$A + B$ is unchanged. Includes explicit constraints (PK, FK) and implicit ones (e.g. total balances – loans = cash-in-hand). DB may be temporarily inconsistent during execution. DBMS guarantees consistency only if the transaction logic is correct.
Isolation	Concurrent execution must be equivalent to some serial execution; intermediate results not visible. For every T_i, T_j : to T_i it appears T_j finished before T_i started or started after T_i finished.	If T_2 runs <i>read(A)</i> , <i>read(B)</i> , <i>print(A+B)</i> between steps 3 and 6, it sees $A + B$ <i>less</i> than it should be. Trivial solution: run serially – but concurrency has big benefits.
Durability	Once committed, changes persist despite failures.	After the user is told the transfer is done, the updates must survive any crash.

Who ensures what

Atomicity & Durability → **recovery manager**; Isolation → **concurrency-control** subsystem; Consistency → **programmer** + integrity constraints.

5 Transaction States (Slides 32–35)

- **Active** – initial state while executing (reads/writes).
- **Partially committed** – after the final statement executes.
- **Failed** – normal execution cannot proceed.
- **Aborted** – rolled back, DB restored to its prior state. Then either *restart* (only if no internal logical error) or *kill*.
- **Committed** – after successful completion.
- **Terminated** (Elmasri Fig. 21.4) – leaves the system after commit or failure.



Implementing Atomicity & Durability – Shadow-database scheme

- The **recovery-management component** implements atomicity and durability.
- All updates are made on a **shadow copy** of the database.
- **db_pointer** is switched to the updated copy only after the transaction reaches *partial commit* and all updated pages are *flushed to disk*. The old copy is then deleted.
- Failure before the switch ⇒ old copy still valid (atomicity). After the switch ⇒ new copy permanent (durability).

6 Concurrent Executions and Schedules (Slides 36–46)

Why allow concurrency?

- **Better CPU and disk utilisation** $\Rightarrow$ higher throughput (one transaction uses the CPU while another does I/O).
- **Reduced average response time** – short transactions need not wait behind long ones.

Concurrency-control schemes = mechanisms to achieve isolation, i.e. control the interaction among concurrent transactions so they don't destroy consistency.

Schedule

A **schedule (history)** is a sequence of instructions giving the chronological order in which instructions of concurrent transactions execute. It must contain *all* instructions of those transactions and *preserve the order within each transaction*. A completed transaction ends with **commit**; a failed one with **abort**.

Formal definition: A schedule S of $T_1, \dots, T_n$ is an ordering of all their operations such that, for each T_i , the operations of T_i appear in S in the same order as in T_i (other transactions' operations may be interleaved).

Number of possible schedules (each T_i has m_i operations):

$$\frac{(m_1 + m_2 + \dots + m_n)!}{m_1! m_2! \dots m_n!}$$

E.g. T_1 with 4 ops and T_2 with 2 ops: $\frac{6!}{4! 2!} = 15$ schedules.

Silberschatz schedules: T_1 transfers \$50 $A \rightarrow B$; T_2 transfers 10% of A to B (start $A = 100$, $B = 200$)

	Schedule	Order	Final A, B	$A + B$
T_1 : read(A); A:=A-50; write(A); read(B); B:=B+50; write(B)	1 (serial)	T_1 then T_2	45, 255	300 ✓
	2 (serial)	T_2 then T_1	40, 260	300 ✓
T_2 : read(A); temp:=A*0.1; A:=A-temp; write(A); read(B); B:=B+temp; write(B)	3 (concurrent)	equivalent to 1	45, 255	300 ✓
	4 (concurrent)	not serializable	50, 210	260 ×

Schedule 3 vs. Schedule 4 (step by step)

Schedule 3 (non-serial, but equivalent to 1)
 T_1 : read(A)=100, A=50, write(A) $\rightarrow$ A=50
 T_2 : read(A)=50, temp=5, A=45, write(A) $\rightarrow$ A=45
 T_1 : read(B)=200, B=250, write(B) $\rightarrow$ B=250
 T_2 : read(B)=250, B=255, write(B) $\rightarrow$ B=255
 Final $A + B = 45 + 255 = 300$ ✓

Schedule 4 (does NOT preserve $A + B$)
 T_1 : read(A)=100, A=50 (not yet written)
 T_2 : read(A)=100, temp=10, A=90, write(A) $\rightarrow$ A=90, read(B)=200
 T_1 : write(A) $\rightarrow$ A=50 (overwrites 90!), read(B)=200, B=250, write(B)
 T_2 : B=200+10=210, write(B) $\rightarrow$ B=210 (overwrites 250!)
 Final $A + B = 50 + 210 = 260$ × (\$40 vanished – lost updates)

Slide 41 correction

The slide's value table for Schedule 4 shows $A = 90$, $B = 250/260$, $A + B = 340/350$. Tracing the operations gives $A = 50$, $B = 210$, $A + B = 260$ (Silberschatz: with $A = 1000$, $B = 2000$ the result is $950 + 2100$). Either way the sum is *not* preserved.

Elmasri Fig. 21.5 – Schedules A–D of T_1, T_2 (compact notation)

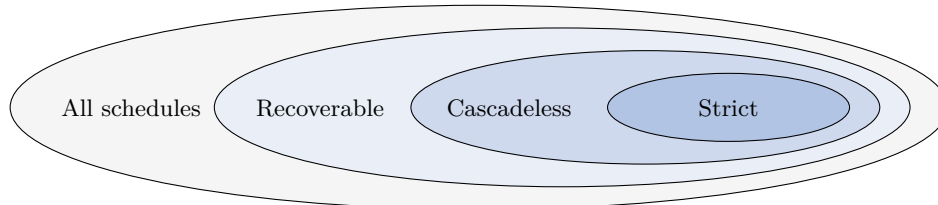
A (serial $T_1 T_2$) $r_1(X); w_1(X); r_1(Y); w_1(Y); r_2(X); w_2(X)$
B (serial $T_2 T_1$) $r_2(X); w_2(X); r_1(X); w_1(X); r_1(Y); w_1(Y)$
C (non-serial) $r_1(X); r_2(X); w_1(X); r_1(Y); w_2(X); w_1(Y)$ (lost update!)
D (non-serial) $r_1(X); w_1(X); r_2(X); w_2(X); r_1(Y); w_1(Y)$

Some schedules are easy to recover from, some not (**recoverability**); some give correct results, some not (**serializability**).

7 Schedules Based on Recoverability (Slides 47–55)

Definitions

- **Recoverable:** no committed transaction ever needs to be rolled back. Formally: *no T commits until every T' that wrote an item T read has committed.*
- **Non-recoverable:** a committed transaction may have to be rolled back – violates *durability*; must not be allowed.
- **Cascading rollback:** an uncommitted T_2 that read from a failed T_1 must also be rolled back.
- **Cascadeless (ACA):** T_2 cannot *read* X until the T_1 that last wrote X has committed/aborted.
- **Strict:** T_2 can neither *read nor write* X until the T_1 that last wrote X has committed/aborted.
- **Blind write:** $w_2(X)$ not preceded by $r_2(X)$.



Strict $\subseteq$ Cascadeless $\subseteq$ Recoverable $\subseteq$ All. If blind writes are *not* allowed: Cascadeless = Strict.

Slide examples

Schedule	Operations	Verdict
A	$r_1(X); w_1(X); r_2(X); w_2(X); c_2; r_1(Y); w_1(Y); c_1$ (or a_1)	Non-recoverable: T_2 read X from T_1 and commits first. If T_1 aborts, committed T_2 is wrong.
B	$r_1(X); w_1(X); r_2(X); w_2(X); r_1(Y); w_1(Y); c_1$ (or a_1); ...	Recoverable (c_2 delayed until after c_1); if T_1 aborts, T_2 must abort $\Rightarrow$ <i>cascading rollback</i> ; not cascadeless .
C	$r_1(X); w_1(X); r_1(Y); w_1(Y); c_1; r_2(X); w_2(X); \dots$	Cascadeless and strict ($r_2(X)$ delayed until T_1 commits; no blind writes).
D	$r_1(X); w_1(X); w_3(X); r_1(Y); w_1(Y); c_1; r_2(X); w_2(X); \dots$	Cascadeless but not strict: blind write $w_3(X)$ before T_1 commits.
E	$r_1(X); w_1(X); r_1(Y); w_1(Y); c_1; w_3(X); r_2(X); w_2(X); \dots$	Strict ($w_3(X)$ delayed until after c_1).

Quick patterns (slide 53): $w_1(X) r_2(X) c_2 c_1 \rightarrow$ **non-recoverable**; $w_1(X) r_2(X) c_1 c_2 \rightarrow$ **recoverable** (not cascadeless – T_2 read uncommitted data); $w_1(X) c_1 r_2(X) c_2 \rightarrow$ **recoverable, cascadeless, strict**.

Slide 53 note

The slide labels “ $w_1(X) r_2(X) c_1 c_2$ ” as *Recoverable Cascadeless*. By the definition on slide 49 it is recoverable but **not** cascadeless, because $r_2(X)$ reads X before T_1 commits.

Property	Condition (if T_i wrote X and T_j is another transaction)
Recoverable	T_j commits only after T_i commits, if T_j read from T_i
Cascadeless	T_j cannot <i>read</i> X until T_i commits
Strict	T_j cannot <i>read or write</i> X until T_i commits

8 Schedules Based on Serializability (Slides 56–67)

- Each transaction on its own is correct (consistency preservation).
- **Serial schedule:** all operations of each transaction execute consecutively, no interleaving. Otherwise **non-serial**. Every serial schedule is **correct**.
- Serial schedules are **not feasible for performance**: no interleaving; long transactions make others wait; CPU idles during I/O.
- **Serializable schedule:** equivalent to *some* serial schedule of the same n transactions. There are $n!$ serial schedules.

Equivalence of schedules

- **Result equivalent:** same final DB state. Not reliable – may hold “by chance”. Fig. 21.6: $S_1: X := X + 10$ and $S_2: X := X \times 1.1$ both give 110 when $X = 100$, but differ for other values.

- **Conflict equivalent:** the relative order of *every pair of conflicting operations* is the same in both schedules (the commonly used definition).

Conflicting operations

Two operations conflict iff (1) they access the **same item**, (2) they belong to **different transactions**, and (3) **at least one is a write**.

T_i	T_j	Conflict?	Type
R(X)	R(X)	No	–
R(X)	W(X)	Yes	read–write
W(X)	R(X)	Yes	write–read
W(X)	W(X)	Yes	write–write

Swapping conflicting operations changes the outcome: $r_1(X); w_2(X) \rightarrow w_2(X); r_1(X)$ makes T_1 read a different value; $w_1(Y); w_2(Y) \rightarrow w_2(Y); w_1(Y)$ changes the final Y . Swapping reads $r_1(Z); r_2(Z)$ changes nothing.

- S is (**conflict**) **serializable** if it is conflict equivalent to some serial schedule S' .
- A serializable schedule is **correct** (same as a serial one) yet gains the **efficiency** of interleaving.
- Serializability is **hard to check at run time** (OS scheduler decides interleaving; transactions continuously start and end).
- **Practical approach:** use **concurrency-control protocols** that guarantee serializability – in most DBMSs, **two-phase locking (2PL)**.

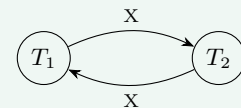
9 Testing Conflict Serializability – Precedence Graph (Slides 68–72)

Algorithm 21.1

1. Create a node for each transaction T_i in S .
2. Edge $T_i \rightarrow T_j$ if T_j does **read_item(X)** *after* T_i does **write_item(X)**. (W–R)
3. Edge $T_i \rightarrow T_j$ if T_j does **write_item(X)** *after* T_i does **read_item(X)**. (R–W)
4. Edge $T_i \rightarrow T_j$ if T_j does **write_item(X)** *after* T_i does **write_item(X)**. (W–W)
5. S is conflict serializable **iff the graph has no cycle**. An equivalent serial order = a **topological sort** of the graph.

Fig. 21.7 – Schedules A–D

- **A:** $T_1 \rightarrow T_2$ (on X) – serial.
- **B:** $T_2 \rightarrow T_1$ (on X) – serial.
- **C:** $r_1(X)$ before $w_2(X)$ gives $T_1 \rightarrow T_2$; $r_2(X)$ before $w_1(X)$ gives $T_2 \rightarrow T_1 \Rightarrow$ **cycle** $\Rightarrow$ **not serializable**.
- **D:** only $T_1 \rightarrow T_2 \Rightarrow$ **serializable**, equivalent to A.



Schedule C: cycle

Fig. 21.8 – three transactions

$T_1 : r(X); w(X); r(Y); w(Y)$ $T_2 : r(Z); r(Y); w(Y); r(X); w(X)$ $T_3 : r(Y); r(Z); w(Y); w(Z)$

Schedule E: $r_2(Z); r_2(Y); w_2(Y); r_3(Y); r_3(Z); r_1(X); w_1(X); w_3(Y); w_3(Z); r_2(X); r_1(Y); w_1(Y); w_2(X)$

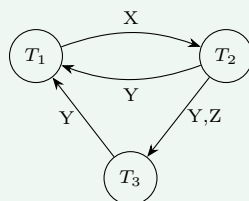
Edges: $T_1 \rightarrow T_2$ (X), $T_2 \rightarrow T_1$ (Y), $T_2 \rightarrow T_3$ (Y, Z), $T_3 \rightarrow T_1$ (Y).

Cycles $T_1 \rightarrow T_2 \rightarrow T_1$ and $T_1 \rightarrow T_2 \rightarrow T_3 \rightarrow T_1 \Rightarrow$ **not serializable** (no equivalent serial schedule).

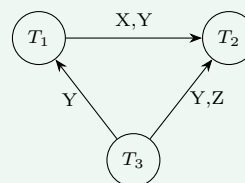
Schedule F: $r_3(Y); r_3(Z); r_1(X); w_1(X); w_3(Y); w_3(Z); r_2(Z); r_1(Y); w_1(Y); r_2(Y); w_2(Y); r_2(X); w_2(X)$

Edges: $T_3 \rightarrow T_1$ (Y), $T_3 \rightarrow T_2$ (Y, Z), $T_1 \rightarrow T_2$ (X, Y). No cycle $\Rightarrow$ **serializable**, equivalent to $T_3 \rightarrow T_1 \rightarrow T_2$.

(f) A graph with only $T_3 \rightarrow T_1$ and $T_3 \rightarrow T_2$ has **two** equivalent serial schedules: $T_3T_1T_2$ and $T_3T_2T_1$.



E: not serializable



F: $T_3 \rightarrow T_1 \rightarrow T_2$

10 View Serializability and Other Equivalences (Slides 73–79)

View equivalence of S and S'

1. Same transactions and same operations.
 2. **Same reads-from:** if $r_i(X)$ in S reads the value written by $w_j(X)$ (or the initial value), it does the same in S' .
 3. **Same final write:** if $w_k(Y)$ is the last write of Y in S , it is also the last in S' .
- View serializable** = view equivalent to some serial schedule. Idea: every read “sees” the same view, and the final DB state is the same.

- Without blind writes, conflict and view serializability are the **same**.
- With blind writes, conflict serializability is **stricter**: *every conflict-serializable schedule is view serializable, but not vice versa*.
- Testing view serializability is NP-hard, so conflict serializability is used in practice.

Slide 77 – view serializable but not conflict serializable

$T_1 : r_1(X); w_1(X), T_2 : w_2(X), T_3 : w_3(X)$ (blind writes in T_2, T_3)

$S_a : r_1(X); w_2(X); w_1(X); w_3(X); c_1; c_2; c_3$

- **Conflict:** $r_1(X)$ before $w_2(X)$ gives $T_1 \rightarrow T_2$; $w_2(X)$ before $w_1(X)$ gives $T_2 \rightarrow T_1 \Rightarrow$ cycle $\Rightarrow$ **not** conflict serializable.
- **View:** compare with serial $T_1 T_2 T_3$: $r_1(X)$ reads the initial value in both, and the final write is $w_3(X)$ in both $\Rightarrow$ **view serializable**.

Semantic (debit–credit) equivalence: addition and subtraction *commute*, so some non-conflict-serializable schedules are still correct.

$S_h : r_1(X); w_1(X); r_2(Y); w_2(Y); r_1(Y); w_1(Y); r_2(X); w_2(X)$ – not conflict serializable ($T_1 \rightarrow T_2$ on X , $T_2 \rightarrow T_1$ on Y). But with the semantics $X := X - 10$; $Y := Y - 20$; $Y := Y + 10$; $X := X + 20$ (debit, debit, credit, credit) the final result is correct.

11 Transaction Support in SQL (Slides 80–86)

- A single SQL statement is always **atomic**.
- **No explicit BEGIN** – a transaction starts implicitly with the first SQL statement.
- Every transaction needs an explicit end: COMMIT or ROLLBACK.
- SET TRANSACTION sets:
 - **Access mode:** READ ONLY or READ WRITE. Default READ WRITE, unless isolation is READ UNCOMMITTED, in which case READ ONLY is assumed.
 - **Diagnostic size n :** number of conditions held in the diagnostic area (run-time feedback).
 - **Isolation level:** READ UNCOMMITTED, READ COMMITTED, REPEATABLE READ, SERIALIZABLE (default). If all transactions use SERIALIZABLE the execution is serializable; lower levels may violate it.

Problems allowed by lower isolation levels:

- **Dirty read** – reading a value written by a transaction that later fails.
- **Non-repeatable read** – another transaction changes a value between two reads (seats on a flight).
- **Phantom** – T_1 reads rows matching a condition (EMP with DNO=5); T_2 inserts a new DNO=5 row; the new row is a **phantom** record.

Isolation level	Dirty read	Non-repeatable read	Phantom
READ UNCOMMITTED	Yes	Yes	Yes
READ COMMITTED	No	Yes	Yes
REPEATABLE READ	No	No	Yes
SERIALIZABLE	No	No	No

Table 21.1 – possible violations per isolation level (“Yes” = may occur).

Sample SQL transaction:

```
EXEC SQL WHENEVER SQLERROR GOTO UNDO;
EXEC SQL SET TRANSACTION READ WRITE DIAGNOSTICS SIZE 5
        ISOLATION LEVEL SERIALIZABLE;
EXEC SQL INSERT INTO EMPLOYEE (FNAME, LNAME, SSN, DNO, SALARY)
        VALUES ('Robert', 'Smith', '991004321', 2, 35000);
```

```

EXEC SQL UPDATE EMPLOYEE SET SALARY = SALARY * 1.1 WHERE DNO = 2;
EXEC SQL COMMIT;
GOTO THE_END;
UNDO:      EXEC SQL ROLLBACK;
THE_END: ...

```

If any statement fails, control jumps to UNDO and the whole transaction (insert + update) is rolled back.

12 Serializability – Silberschatz View (Slides 87–90)

- **Assumption:** each transaction preserves consistency $\Rightarrow$ serial execution preserves consistency.
- A (possibly concurrent) schedule is **serializable** if equivalent to a serial schedule. The two forms are **conflict serializability** and **view serializability**.
- Simplified view: ignore everything except **read** and **write**; arbitrary local computation may happen between them.
- Instructions l_i and l_j of T_i, T_j **conflict** iff some item Q is accessed by both and **at least one writes Q** : read/read no; read/write, write/read, write/write yes.
- A conflict forces a (logical) temporal order. **Consecutive non-conflicting instructions can be swapped** without changing the result.

Conflict serializability

If S can be transformed into S' by a series of **swaps of non-conflicting instructions**, then S and S' are **conflict equivalent**. S is **conflict serializable** if it is conflict equivalent to a serial schedule.

Swapping example – Schedule 3 becomes Schedule 1

Schedule 3 (reads/writes only): $r_1(A) w_1(A) r_2(A) w_2(A) r_1(B) w_1(B) r_2(B) w_2(B)$

1. $r_1(B)$ and $w_2(A)$ touch different items, so swap them; then swap $r_1(B)$ with $r_2(A)$.

2. Likewise move $w_1(B)$ before $w_2(A)$ and $r_2(A)$.

3. Result: $r_1(A) w_1(A) r_1(B) w_1(B) r_2(A) w_2(A) r_2(B) w_2(B) = \text{serial } T_1 T_2$ (Schedule 1).

So Schedule 3 is **conflict serializable**. Schedule 4 is not: $r_1(A)$ before $w_2(A)$ ($T_1 \rightarrow T_2$) and $r_2(A)$ before $w_1(A)$ ($T_2 \rightarrow T_1$) form a cycle.

13 One-Page Revision

Transaction	Logical unit of DB work; ops read/write; ends in commit or abort.
ACID	Atomicity (all/none), Consistency (valid $\rightarrow$ valid), Isolation (as if serial), Durability (survives failure).
States	Active $\rightarrow$ Partially committed $\rightarrow$ Committed; Active/Partially committed $\rightarrow$ Failed $\rightarrow$ Aborted (restart/kill).
Concurrency problems	Lost update, dirty read, incorrect summary, unrepeatable read (+ phantom).
Failure causes	Crash, transaction error, exception, CC abort, disk failure, catastrophe.
Recovery ops	undo (one write), redo (committed write).
#schedules	$(\sum m_i)! / \prod m_i!$.
Recoverability	Recoverable $\supseteq$ Cascadeless $\supseteq$ Strict; non-recoverable not allowed.
Conflict	Same item, different transactions, at least one write.
Conflict-serializable test	Precedence graph; acyclic $\Leftrightarrow$ serializable; topological order = serial order.
View serializable	Same reads-from + same final writes; conflict-ser. $\subset$ view-ser. (differ only with blind writes).
SQL	Implicit begin; COMMIT/ROLLBACK; SET TRANSACTION access mode, diagnostics size, isolation level (default SERIALIZABLE).
Isolation table	RU: all 3 anomalies; RC: no dirty read; RR: only phantoms; SER: none.